

Golden Section Search Method for Univariate Optimization

Sulagna Roy

2026-09-13

Contents

Overview	1
The Objective Function	1
The Golden Section Search Function	2
Explanation, step by step	3
Running the Search	4
Interpreting the result	5
Visualizing the Convergence	6
Reading the plot	6
Summary	7

Overview

This document explains an implementation of the **golden section search** method, a derivative-free technique for finding the minimizer of a *unimodal* function over a closed interval $[a, b]$. Like Fibonacci search, it works by shrinking a bracketing interval guaranteed to contain the minimum, evaluating the function at two interior points and discarding the sub-interval that cannot contain the minimizer. Unlike Fibonacci search, it uses a single fixed ratio — the **golden ratio conjugate**, $\tau \approx 0.618$ — at every iteration, so the number of iterations does not need to be precomputed; the loop simply runs until the bracket width is smaller than a chosen tolerance.

The Objective Function

```
# Objective function
f <- function(x) {
  (x - 2)^2 * (x - 8)^2
}
```

The function is

$$f(x) = (x - 2)^2(x - 8)^2,$$

algebraically the same shape as $[(x-2)(x-8)]^2$: it touches zero at $x = 2$ and $x = 8$ and is positive elsewhere, giving **two local minima** separated by a local maximum. On the search interval $[1, 5]$ used below, only the minimum at $x = 2$ lies inside the bracket, so that is the one found.

The Golden Section Search Function

```
# Golden Section Search
golden_search <- function(f, a, b, tol = 1e-4) {

  tau <- (sqrt(5) - 1) / 2 # 0.618...

  history <- data.frame(
    iter = 0,
    a = a,
    b = b
  )

  x1 <- b - tau * (b - a)
  x2 <- a + tau * (b - a)

  f1 <- f(x1)
  f2 <- f(x2)

  iter <- 0
  while ((b - a) > tol) {
    iter <- iter + 1

    if (f1 > f2) {

      a <- x1
      x1 <- x2
      f1 <- f2

      x2 <- a + tau * (b - a)
      f2 <- f(x2)

    } else {

      b <- x2
      x2 <- x1
      f2 <- f1

      x1 <- b - tau * (b - a)
      f1 <- f(x1)
    }
  }
  history <- rbind(
    history,
```

```

    data.frame(iter = iter, a = a, b = b)
  )
}

list(
  xmin = (a + b) / 2,
  interval = c(a, b),
  history = history
)
}

```

Explanation, step by step

1. The golden ratio conjugate

```
tau <- (sqrt(5) - 1) / 2 # 0.618...
```

$\tau = \frac{\sqrt{5}-1}{2} \approx 0.618$ is the reciprocal of the golden ratio φ . It has the special self-similar property $\tau^2 = 1 - \tau$, which is exactly what allows one interior point to be reused between iterations without shifting position relative to the new bracket — the same mechanism Fibonacci search approximates using ratios of Fibonacci numbers, except here it is a single constant rather than a changing sequence.

2. Initializing the history and interior points

```

history <- data.frame(iter = 0, a = a, b = b)
x1 <- b - tau * (b - a)
x2 <- a + tau * (b - a)
f1 <- f(x1)
f2 <- f(x2)

```

`history` records the bracket at iteration 0 (the original interval) before any shrinking happens. Two interior points are placed symmetrically inside $[a, b]$:

$$x_1 = b - \tau(b - a), \quad x_2 = a + \tau(b - a),$$

with $x_1 < x_2$. Both points are evaluated once up front (`f1`, `f2`).

3. The main loop

```

iter <- 0
while ((b - a) > tol) {
  iter <- iter + 1
  if (f1 > f2) {
    a <- x1; x1 <- x2; f1 <- f2
    x2 <- a + tau * (b - a); f2 <- f(x2)
  } else {
    b <- x2; x2 <- x1; f2 <- f1
    x1 <- b - tau * (b - a); f1 <- f(x1)
  }
  history <- rbind(history, data.frame(iter = iter, a = a, b = b))
}

```

Unlike Fibonacci search's `for` loop over a precomputed number of steps, this is a `while` loop that runs until the raw bracket width `b - a` drops below `tol`. Each pass:

- **If $f(x_1) > f(x_2)$:** the minimum cannot lie left of x_1 , so `a <- x1`. The old x_2 slides into the x_1 role (reusing `f2` as the new `f1`), and a fresh x_2 is computed with the same formula `a + tau * (b - a)` against the *new*, narrower interval, needing one new evaluation of `f`.
- **Otherwise:** symmetric — `b <- x2`, the old x_1 becomes the new x_2 (reusing `f1` as `f2`), and a fresh x_1 is computed via `b - tau * (b - a)`, again one new evaluation.

Every iteration therefore costs exactly **one** new function evaluation, the same efficiency Fibonacci search achieves, but here obtained from the fixed-point property of τ rather than from a precomputed sequence. The updated bracket is appended to `history` after each pass.

4. Returning the result

```
list(  
  xmin = (a + b) / 2,  
  interval = c(a, b),  
  history = history  
)
```

When the loop exits, `b - a \le tol`, so the bracket is tiny. The function returns the midpoint of that final bracket as `xmin`, the final `interval` itself, and the full `history` of every iteration's bracket for inspection or plotting.

Running the Search

```
# Run Golden Section Search  
result <- golden_search(f, a = 1, b = 5, tol = 1e-4)  
  
print(result)  
  
## $xmin  
## [1] 1.999998  
##  
## $interval  
## [1] 1.999967 2.000029  
##  
## $history  
##      iter      a      b  
## 1      0 1.000000 5.000000  
## 2      1 1.000000 3.472136  
## 3      2 1.000000 2.527864  
## 4      3 1.583592 2.527864  
## 5      4 1.583592 2.167184  
## 6      5 1.806504 2.167184  
## 7      6 1.944272 2.167184  
## 8      7 1.944272 2.082039  
## 9      8 1.944272 2.029417  
## 10     9 1.976794 2.029417  
## 11    10 1.976794 2.009317
```

```
## 12 11 1.989217 2.009317
## 13 12 1.996894 2.009317
## 14 13 1.996894 2.004572
## 15 14 1.996894 2.001639
## 16 15 1.998707 2.001639
## 17 16 1.998707 2.000519
## 18 17 1.999399 2.000519
## 19 18 1.999827 2.000519
## 20 19 1.999827 2.000255
## 21 20 1.999827 2.000091
## 22 21 1.999928 2.000091
## 23 22 1.999928 2.000029
## 24 23 1.999967 2.000029
```

Interpreting the result

On $[a, b] = [1, 5]$ with `tol = 1e-4`:

- `xmin` ≈ 1.999998 — essentially exact, since the true minimizer is $x = 2$.
- `interval` $\approx [1.999967, 2.000029]$ — a final bracket of width about 6×10^{-5} , comfortably under the 10^{-4} tolerance.
- `history` has 24 rows (iterations 0 through 23), meaning the `while` loop executed 23 times before `b - a` fell below `tol`. As with Fibonacci search, a rises and b falls monotonically, but here the shrink factor per step is the constant $\tau \approx 0.618$, so the bracket width shrinks geometrically rather than by a step-dependent Fibonacci ratio.

```
h <- result$history
h
```

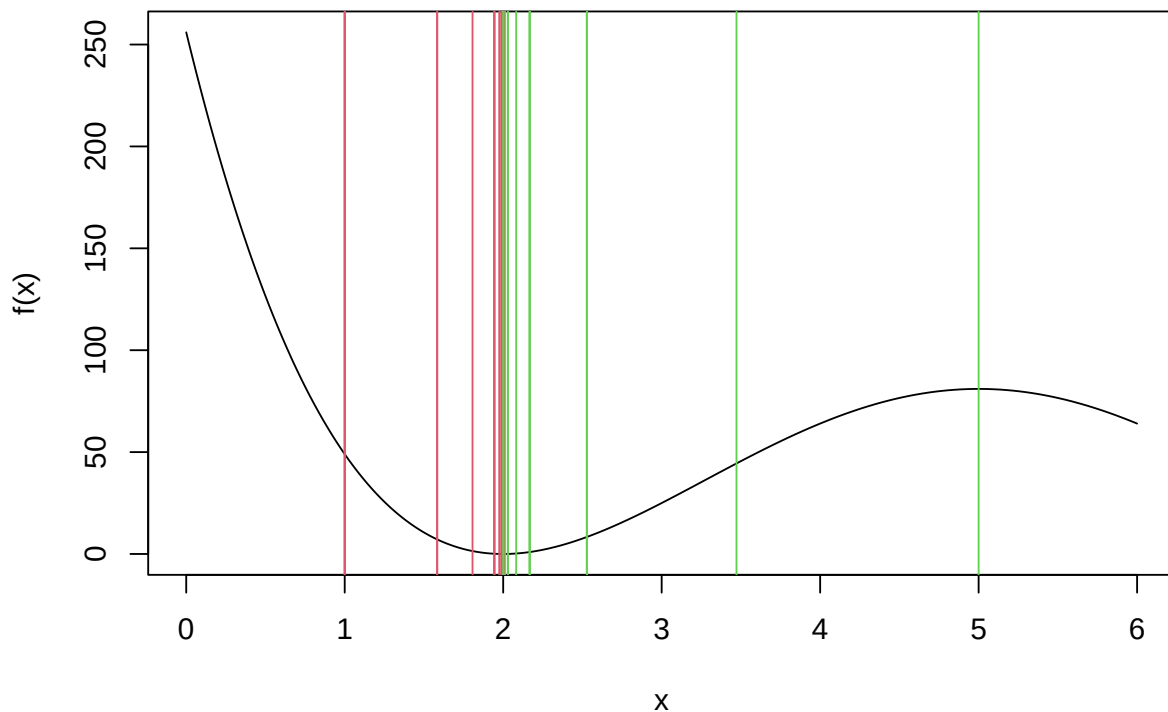
```
##      iter      a      b
## 1      0 1.000000 5.000000
## 2      1 1.000000 3.472136
## 3      2 1.000000 2.527864
## 4      3 1.583592 2.527864
## 5      4 1.583592 2.167184
## 6      5 1.806504 2.167184
## 7      6 1.944272 2.167184
## 8      7 1.944272 2.082039
## 9      8 1.944272 2.029417
## 10     9 1.976794 2.029417
## 11    10 1.976794 2.009317
## 12    11 1.989217 2.009317
## 13    12 1.996894 2.009317
## 14    13 1.996894 2.004572
## 15    14 1.996894 2.001639
## 16    15 1.998707 2.001639
## 17    16 1.998707 2.000519
## 18    17 1.999399 2.000519
## 19    18 1.999827 2.000519
## 20    19 1.999827 2.000255
## 21    20 1.999827 2.000091
## 22    21 1.999928 2.000091
## 23    22 1.999928 2.000029
## 24    23 1.999967 2.000029
```

Visualizing the Convergence

```
u <- seq(0, 6, length = 500)
plot(u, f(u), type = 'l',
     main = "Golden Section Search: Shrinking Bracket over f(x) = (x-2)^2(x-8)^2",
     xlab = "x", ylab = "f(x)")

for (i in 1:nrow(h)) {
  abline(v = h[i, 2], col = 2) # left endpoint a, in red
  abline(v = h[i, 3], col = 3) # right endpoint b, in green
}
```

Golden Section Search: Shrinking Bracket over $f(x) = (x-2)^2(x-8)^2$



Reading the plot

The black curve is $f(x) = (x-2)^2(x-8)^2$ over $x \in [0, 6]$, dipping to zero at $x = 2$ and beginning to rise again toward the second (unreached) minimum near $x = 8$, just visible climbing at the right edge of the plot.

- **Red lines** are the left endpoint a (column 2 of h) at each iteration.
- **Green lines** are the right endpoint b (column 3 of h) at each iteration.

The first red/green pair spans the full starting interval $[1, 5]$; with each successive iteration a new red line sits a little further right and a new green line a little further left, so the bundle of lines visibly narrows around

$x = 2$. By the final iterations the red and green lines are packed together indistinguishably at the bottom of the left dip, matching the converged `interval` reported above.

Summary

This script implements golden section search:

1. It uses the single constant $\tau = (\sqrt{5} - 1)/2$ to place two interior points inside the bracket at every iteration.
2. Its self-similarity property lets one evaluation be reused each step, so only one new evaluation of `f` is needed per iteration.
3. A `while` loop shrinks the bracket geometrically by factor τ until its width is below `tol`, with no need to precompute the iteration count.
4. It records the bracket history for plotting and returns the midpoint of the final tiny bracket as the estimated minimizer.

For $f(x) = (x - 2)^2(x - 8)^2$ on $[1, 5]$, the method converges to $x \approx 2.000000$ in 23 iterations, matching the true local minimizer to well within the requested tolerance.